TECHNICAL REFERENCE

GitHub Actions Workflow for Terraform Project

Security scanning, linting, cost estimation, and drift detection with GitHub Actions — a comprehensive guide to securing and automating Terraform CI/CD pipelines.

April 21, 2026 | Infrastructure as Code | DevSecOps

CONTENTS

1. Checkov Security Scanning
2. TFLint — Terraform Linting
3. Terrascan
4. TFSec Static Analysis
5. InfraCost — Cost Estimation
6. Tool Comparison (2026)
7. Drift Detection

CHAPTER 1

Terraform Security Scanning with Checkov and GitHub Actions

Introduction

Terraform is widely used for managing cloud infrastructure, but misconfigurations can introduce security vulnerabilities. Terraform enables developers to create cloud resources via a few lines of code. Hence, these developers must write code that is easy to understand and **follows security best practices**. Moreover, following best practices becomes a habit if there is a system to keep that in check. **Checkov** is a proven static analysis tool that checks for **standard best practices** in your Terraform code.

What is Checkov?

Checkov is an open-source static analysis tool developed by **Bridgecrew** that scans **Terraform, CloudFormation, Kubernetes, and Ansible** configurations for security vulnerabilities **before deployment**.

Why Checkov?

- **Static Code Analysis** — Detects misconfigurations before resources are deployed.
- **Multi-Cloud & IaC Support** — Works with Terraform, Kubernetes, Ansible, and more.
- **Pre-Built Security Policies** — Leverages an extensive security ruleset.
- **CI/CD Integration** — Easily integrates into GitHub Actions, Jenkins, and GitLab CI/CD.
- **Custom Policy Support** — Allows users to define their own security policies.

> **IDEAL CHOICE**
>
> This makes Checkov an **ideal choice** for automating Terraform security scanning inside CI/CD pipelines.

What Security Issues Can Checkov Detect?

Code scanning is auto-enabled once you add the Checkov scanning YAML file to your repository, provided your repository is **open source (public repository)**. Checkov helps identify a **wide range of security misconfigurations**, including:

- **IAM Misconfigurations** — Detects overly permissive IAM policies, missing least privilege enforcement, and unrestricted access to sensitive resources.
- **S3 Bucket Security Risks** — Flags public bucket exposures, missing encryption, lack of versioning, and insecure ACLs.
- **Network Security Issues** — Identifies security group misconfigurations, open inbound ports, and missing VPC peering restrictions.
- **Encryption & Secrets Handling** — Ensures sensitive data is encrypted at rest and in transit, flags hardcoded credentials, and missing KMS integrations.
- **Compute & Infrastructure Risks** — Detects unpatched AMIs, exposed EC2 instances, and missing runtime security settings in Kubernetes & Docker.

Why GitHub Actions for Terraform Security?

GitHub Actions is a widely used CI/CD automation tool that integrates seamlessly with Terraform and Checkov.

- **Built-in CI/CD** — GitHub Actions enables fully automated security scans.

- **Customizable Workflows** — Supports scripting to tailor security workflows.
- **Security-First Approach** — Integrates security scanning without disrupting Terraform deployments.
- **On-Demand Execution** — Checkov scans can be triggered per PR or scheduled for full repo scans.

Automating Terraform Security in CI/CD

Two Terraform scanning pipelines have been built using GitHub Actions + Checkov:

- **Full Repository Scans** — Scheduled scans for the entire Terraform codebase.
- **PR-Specific Scans** — Scans only modified files in a PR before merging.

GitHub Actions Workflow for Full Terraform Repository Scans

Workflow Steps:

- Checkout Terraform repository from GitHub.
- Run Checkov security scans across all Terraform files.
- Generate scan reports in JSON format.
- Upload reports to AWS S3 for tracking and historical analysis.

> **OUTPUT**
>
> Reports stored in AWS S3, ready for review.

PR-Specific Scans with GitHub Actions & Checkov

Enhancement: Instead of scanning the full repository, this pipeline only scans Terraform files changed in a PR.

- Detects which Terraform files changed in the PR.
- Runs Checkov scan only on modified files.
- Posts summary results as a comment in the PR for reviewers.
- Allows developers to fix issues before merging.

Handling Multi-Account Terraform Scans

In a multi-account environment, Checkov can be configured to scan Terraform code separately per AWS account, ensuring each environment is validated independently.

- **Account-Specific Validation** — Ensures security scanning is tailored for staging, production, and other environments.
- **Granular Module Scanning** — Targets specific Terraform modules (e.g., `/alb`) instead of scanning the full repository.
- **Excluding Deprecated Code** — The `archive/` directory contains deprecated Terraform code that is no longer in use. Skipping it avoids unnecessary alerts and ensures that only active infrastructure is scanned.

Conclusion

By integrating Checkov into GitHub Actions, we achieve:

- Full repo scans for complete security auditing
- PR-specific scans for proactive vulnerability detection
- Automated CI/CD security enforcement

CHAPTER 2

Enhancing Your Terraform Code with TFLint

Detecting errors at early stages is good in any development environment, and Terraform is no different. TFLint catches errors, enforces best practices, and validates AWS-specific Terraform configurations before they cause problems. Terraform's built-in `validate` command checks syntax, but it doesn't catch common mistakes — using an invalid instance type? It won't tell you until apply time. TFLint fills that gap by catching AWS-specific errors and enforcing best practices at lint time.

What is TFLint?

TFLint is a popular open-source linter and static analysis tool designed explicitly for Terraform. It performs automated checks on Terraform configurations to identify potential issues, errors, and violations of best practices. TFLint helps maintain code quality, consistency, and reliability.

TFLint automatically scans `.tf` files and reports potential issues by analyzing Terraform code for stylistic errors, security problems, or provider-specific issues before deployment. You can extend functionality using plugins for cloud providers like AWS, Azure, or Google Cloud. Configuration is done via a `.tflint.hcl` file.

The .tflint.hcl Configuration File

The `.tflint.hcl` file customizes TFLint's behavior with the following options:

1. **Ruleset selection** — Predefined sets of rules focusing on specific aspects such as cloud-specific rules, security rules, or best practice rules.
2. **Rule customization** — Change the severity level (e.g., error, warning, info) for specific rules or exclude certain rules.
3. **Ignore rules** — Specify rules that should be ignored for specific files or directories.
4. **Plugin configuration** — Enable or disable specific plugins and set their options.

Example AWS plugin configuration:

```
plugin "aws" {
  enabled = true
  region  = "us-east-1"
}
```

TFLint vs Terraform Validate

They are complementary, not replacements:

Terraform validate catches	TFLint additionally catches
Syntax errors	Invalid AWS resource attribute values (instance types, regions, etc.)
Invalid block types	Deprecated resource configurations
Missing required arguments	Naming convention violations

Type mismatches in variables	Missing variable descriptions
	AWS-specific best practice violations

Use both in your pipeline — `terraform validate` first, then TFLint:

```
# Run both checks in sequence
terraform validate && tflint
```

Running TFLint

```
# Lint current directory
$ tflint

# Lint a specific module directory
$ tflint --chdir=./modules/my-module

# Recursively lint all subdirectories (for monorepos)
$ tflint --recursive
```

Combining TFLint with Other Tools

A typical validation pipeline runs in this order:

1. `terraform fmt` — formatting check
2. `terraform validate` — syntax and type checks
3. `tflint` — AWS-specific linting
4. `checkov` — security scanning
5. `terraform plan` — actual plan review

Summary

KEY TAKEAWAY

TFLint catches a class of errors that no other tool in the Terraform ecosystem handles. Invalid instance types, deprecated configurations, and naming convention violations are exactly the kind of things that slip through code review but cause problems at apply time. Setting it up takes five minutes, and it saves you from countless failed deployments.

CHAPTER 3

Terrascan — Static Code Analysis for Terraform

DEPRECATED

Terrascan was **archived in November 2025**. The GitHub repository is now read-only with no new releases, patches, or issue tracking. Existing Terrascan users should plan a migration to an actively maintained alternative.

Introduction

Terrascan is a static code analyzer for Infrastructure as Code that allows you to:

- Seamlessly scan infrastructure as code for misconfigurations.
- Monitor provisioned cloud infrastructure for configuration changes that introduce posture drift.
- Detect security vulnerabilities and compliance violations.
- Mitigate risks before provisioning cloud native infrastructure.
- Run locally or integrate with your CI/CD pipeline.

What is Terrascan?

Terrascan is a flexible and powerful open-source static code analysis tool designed for scanning infrastructure as code (IaC) templates and configurations. It uses Rego policies from **Open Policy Agent (OPA)** to enforce security best practices.

Terrascan Highlights

- Supports over **500 built-in policies**.
- Scans Terraform HCL files.
- Integrates with CI/CD pipelines.
- Detects AWS, Azure, GCP, and Kubernetes misconfigurations.

Key Benefits

- Fast and free.
- Over 500 built-in policies.
- Works locally and in CI/CD.
- Secures Terraform, Kubernetes, and more.

Scan Commands

```
# Scan current directory (defaults to Terraform)
$ terrascan scan

# Scan AWS IaC configurations
$ terrascan scan -t aws
$ terrascan scan -i terraform -t aws --find-vuln

# Scan a specific file
$ terrascan scan -t aws -f main.tf

# Scan Kubernetes manifests
$ terrascan scan -i k8s

# Scan remote Git repositories
$ terrascan scan -t aws -r git -u git@github.com:accurics/KaiMonkey.git//terraform/

# Scan Helm Charts
$ terrascan scan -i helm

# Scan Dockerfiles
$ terrascan scan -i docker
```

Terrascan vs Checkov

On the face of it, Terrascan and Checkov are very similar — both are open-source static code analysis tools for IaC security and compliance scanning. Key differences:

- Checkov is built using **Python**; Terrascan uses **Go**.
- Custom policies in Terrascan are written using **Rego**; Checkov supports Python or YAML.
- If you prefer using Rego for custom policies, Terrascan is a good choice (while it was active).

Summary

Terrascan was a versatile open-source static code analysis tool specialized in scanning IaC templates and configurations, effectively detecting security vulnerabilities and compliance breaches. As it is now archived, teams should migrate to Checkov or Trivy.

CHAPTER 4

Securing Your Terraform Code with TFSec

> **DEPRECATED**
>
> TFSec has been **fully deprecated and archived**. All tfsec checks have been migrated into **Trivy**. TFSec does not support Terraform 1.5+ features. To migrate, replace `tfsec .` with `trivy config .` — all check IDs remain unchanged in Trivy.

What is tfsec?

tfsec is an open-source tool that performs static analysis of your Terraform code. It checks for security vulnerabilities, potential misconfigurations, and deviations from best practices. By integrating tfsec into your development process, you can catch and fix security issues early, before they make it to production.

Key Features of TFSec

- **Deep Analysis** — TFSec digs into your Terraform code to identify problems that might go unnoticed, such as open security groups, hard-coded secrets, or overly permissive IAM roles.
- **Customizable Rules** — You can tailor tfsec's checks to suit your project's specific needs or compliance requirements.
- **Integration Friendly** — Tfsec can easily integrate with your CI/CD pipeline.
- **Comprehensive Scanning** — Supports multiple cloud providers including AWS, Azure, and GCP.
- **Scanning Modules and Variables** — Scans both local and remote modules, not just main Terraform files.
- **Detailed Reporting** — Provides detailed reports and recommendations in multiple output formats.
- **Policy Customization** — Define custom policies and checks using JSON or YAML.

Basic Usage

```
# Scan the current directory
$ tfsec .

# Scan a specific directory
$ tfsec ./environments/production

# Output results in JSON format
$ tfsec . --format json
```

Understanding Severity Levels

Severity	Description
CRITICAL	Issues that likely expose infrastructure to direct attack (e.g., public S3 buckets, unrestricted security groups)
HIGH	Significant security weaknesses (e.g., missing encryption, weak IAM policies)
MEDIUM	Misconfigurations that weaken your security posture (e.g., missing logging, default VPCs)

LOW	Best practice recommendations (e.g., missing descriptions, resource tagging)

```
# Only show critical and high severity issues
$ tfsec . --minimum-severity HIGH
```

Generating Reports

```
# JSON report for programmatic processing
$ tfsec . --format json --out report.json

# JUnit XML for CI/CD integration
$ tfsec . --format junit --out report.xml

# CSV for spreadsheet analysis
$ tfsec . --format csv --out report.csv

# SARIF for GitHub Security integration
$ tfsec . --format sarif --out report.sarif
```

Understanding the tfsec Output

The output summary provides these counts:

1. **Passed** — Number of checks passed; no action required.
2. **Ignored** — Checks explicitly skipped.
3. **Critical / High / Medium / Low** — Failed checks counted by severity level.

The "counts" section describes "code coverage":

1. **Modules downloaded** — Externally downloaded modules counted.
2. **Modules processed** — Sum of all externally downloaded modules + 1.
3. **Blocks processed** — Number of Terraform blocks processed by tfsec.
4. **Files read** — All configuration files scanned.

Ignoring tfsec Checks

To ignore a specific check, add a comment at the top of the resource block:

```
#tfsec:ignore:<check-id>
```

Custom Rules

If built-in rules do not cover your organization's requirements, you can write custom rules. Save them in `.tfsec/custom_checks.json` and tfsec will pick them up automatically.

For custom checks in YAML, create a subdirectory named `.tfsec` and a YAML file named `_tfchecks.YAML` within it.

Tfsec Alternatives (2026)

Since tfsec is now deprecated, consider these alternatives:

- **Trivy** — Drop-in replacement; all tfsec check IDs remain unchanged.
- **Checkov** — Broader Python-based IaC scanner with 1,000+ policies.
- **KICS** — OPA/Rego-based, 2,400+ policies across 22+ platforms.

CHAPTER 5

InfraCost — Estimate Cloud Costs with Terraform

Cloud infrastructure costs can spiral quickly when teams deploy resources without visibility into pricing. **Infracost** solves this by showing you the cost impact of Terraform changes before you apply them.

What is Infracost?

Infracost is a tool designed to estimate the cost of your infrastructure based on the infrastructure as code (IaC) configuration you have defined. It analyzes the resources defined in your IaC files and provides a detailed overview of costs, making comparisons between the current state of your infrastructure and the new version you want to deploy.

How does Infracost Work?

Infracost is integrated with an Infrastructure as Code tool such as Terraform to provide cost estimates. It compares the current state against a Terraform plan and retrieves cloud prices to calculate costs. It can be integrated into a CI/CD pipeline, and you can take advantage of Open Policy Agent (OPA) policies to make decisions about deploying resources based on costs.

Infracost Use Cases

- **Pre-deployment cost estimation** — See what you will pay before deploying your infrastructure.
- **Cost optimization** — Find out what resources are the most expensive and decide if you need them, or downgrade to a smaller shape.
- **CI/CD integration** — Add cost estimation checks in the code review process.
- **Cost decision-based deployments** — Use OPA policies to set a monthly cost threshold and deny runs that exceed it.
- **Reporting** — Generate detailed reports related to costs that can be shared with key stakeholders.
- **Multi-cloud cost management** — Compare costs across multiple cloud providers.

Installation and Setup

```
# 1. Install the Infracost CLI, then verify:
$ infracost --version

# 2. Register for a Free API Key:
$ infracost auth login

# 3. Retrieve your API Key:
$ infracost configure get api_key

# 4. Set the API key as an environment variable:
$ export INFRACOST_API_KEY=your_api_key_here

# 5. Run your first cost estimate:
$ cd my-terraform-project
$ infracost breakdown --path .
```

Viewing Cost Differences (Before/After Changes)

A key feature of Infracost is its ability to compare costs between a baseline configuration and modified code:

```
# Generate a cost diff
$ infracost diff --path .
$ infracost diff --path . --compare-to old_config.json
```

Using Infracost with Terraform Plan

For the most accurate estimates, use a Terraform plan file:

```
# Generate a Terraform plan
$ terraform plan -out=tfplan.binary

# Convert to JSON
$ terraform show -json tfplan.binary > plan.json

# Run Infracost against the plan
$ infracost breakdown --path plan.json
```

Understanding Cost Output Formats

```
# Detailed output with all components
$ infracost breakdown --path . --show-skipped

# JSON output for programmatic use
$ infracost breakdown --path . --format json > costs.json

# HTML report
$ infracost breakdown --path . --format html > cost-report.html

# Table format (default)
$ infracost breakdown --path . --format table
```

Cost Policies

Set cost thresholds to catch expensive changes:

```
# Fail if monthly cost exceeds a threshold
$ infracost breakdown --path . --format json | \
  jq -e '.totalMonthlyCost | tonumber < 1000' || \
  echo "WARNING: Monthly cost exceeds $1000"

# Check cost increase percentage
$ infracost diff --path . --format json | \
  jq -e '.diffTotalMonthlyCost | tonumber < 100' || \
  echo "WARNING: Cost increase exceeds $100/month"
```

Multi-Project Cost Summary

```
# Generate individual breakdowns
$ infracost breakdown --path environments/production --format json > prod-costs.jso
$ infracost breakdown --path environments/staging --format json > staging-costs.jso

# Combine into a summary
$ infracost output --path "prod-costs.json" --path "staging-costs.json" --format ta
```

Compare Cost Changes (Pull Request Workflow)

One of Infracost's most powerful features is the ability to compare infrastructure cost before and after code changes:

Before the Change — Create a Baseline

```
$ infracost breakdown --path . --format json --out-file infracost-base.json
```

This scans the current Terraform state, calculates the monthly cost estimate, and saves the cost structure to `infracost-base.json`.

After the Change — Run the Diff

```
$ infracost diff --path . --compare-to infracost-base.json
```

This reads the new version of the code, compares it against the baseline, and outputs a human-readable diff showing:

1. What changed
2. New monthly cost estimate
3. Increase or decrease in cost
4. Which specific resources were affected

> **PR WORKFLOW EXAMPLE**
>
> In a team environment using GitHub/GitLab, the `infracost diff` step runs in the CI/CD pipeline and adds a comment to the pull request, like:
> *"Cost estimate change: +$40/month — aws_instance.app added, aws_db_instance upgraded from db.t3.medium to db.t3.large"*

Best Practices

- Run Infracost on every pull request to catch cost increases early.
- Provide usage estimates for resources with usage-based pricing.
- Set cost thresholds to alert on significant increases.
- Review the cost breakdown before approving any `terraform apply`.
- Use the configuration file for multi-project setups.
- Keep Infracost updated for the latest pricing data.
- Compare costs between environments to identify optimization opportunities.

Conclusion

InfraCost brings cost visibility to your Terraform workflow, enabling teams to make informed decisions about infrastructure changes. By running cost estimates before applying changes, you can prevent budget surprises and optimize cloud spending. Whether used locally or integrated into CI/CD pipelines, Infracost is an essential tool for any team managing cloud infrastructure with Terraform.

CHAPTER 6

IaC Security Tool Comparison — 2026

CRITICAL UPDATES

tfsec has been fully deprecated and archived — all checks migrated to Trivy. **Terrascan** was archived in November 2025. In 2026, the real comparison for a production Terraform project is **Checkov vs Trivy vs KICS**.

Tool-by-Tool Summary

Checkov — Best for Pure Terraform Depth

Checkov is the most widely adopted open-source IaC scanner, with over **80 million downloads**, **1,000+ built-in policies**, and graph-based cross-resource checks that no other free tool matches. Palo Alto Networks maintains it as part of Prisma Cloud, but the standalone CLI runs without any paid subscription.

Beyond simple attribute checks, Checkov builds a graph of IaC resources and analyzes dependencies and connections to catch deeper configuration issues. It easily plugs into GitHub, GitLab, pre-commit hooks, IDEs, and supports output formats like JSON, JUnit-XML, and SARIF. Custom policies are written in **Python or YAML** — no Rego required.

Trivy (tfsec Successor) — Best for Broad Coverage

Trivy covers Terraform, CloudFormation, Kubernetes, Helm, Dockerfiles, ARM, and Ansible. It absorbed tfsec and carries its full check library. However, Trivy has fewer IaC-specific policies than Checkov and lacks graph-based cross-resource checks.

If your main concern is Terraform misconfiguration depth, Checkov still has the edge. If you want one tool for IaC plus containers and dependencies, Trivy is the better pick.

KICS — Best for OPA/Rego Teams

KICS has **2,400+ Rego queries** and uses OPA/Rego for its policy engine. It covers 22+ platforms and is a good fit if your team is already invested in OPA.

Recommendation Matrix

Scenario	Best Tool
Terraform is your primary IaC	**Checkov** Recommended

You also manage containers / K8s / Helm	**Trivy**
Team uses OPA/Rego for custom policies	**KICS**
Multi-IaC + widest platform support	**KICS**
Coming from tfsec	**Trivy** (drop-in replacement)

2026 RECOMMENDATION

For most production Terraform projects in 2026, **Checkov** is the top recommendation — it has the deepest Terraform policy coverage, graph-based analysis, active maintenance by Palo Alto Networks, and the largest community. If you need broader security coverage beyond IaC (containers, secrets, SBOMs), add **Trivy** alongside it.

CHAPTER 7

Terraform Drift Detection with GitHub Actions

Introduction

Infrastructure drift occurs when your live infrastructure no longer matches what your infrastructure code defines. Manual changes through cloud consoles, auto-scaling adjustments, and updates from other automation tools all create discrepancies between your defined state and actual resources. These changes accumulate over time, creating a gap between what your code says should exist and what actually exists in your cloud accounts.

Most teams discover drift only when something breaks. Without systematic drift detection, these surprises become increasingly common as the infrastructure scales and more teams interact with cloud resources. Catching drift early prevents these disruptions and maintains a reliable infrastructure footprint.

What is Terraform Drift?

Terraform manages infrastructure through three states:

- The configuration in your `.tf` files
- The recorded state in `terraform.tfstate`
- The actual state of resources in your cloud provider

Drift happens when actual resources diverge from what Terraform expects based on its state file. This divergence occurs constantly in production environments. For example, when:

- Engineers adjust load balancer health checks while troubleshooting
- Auto-scaling groups modify instance counts based on traffic
- Cloud providers update managed services with patches
- Security teams apply firewall rules through the console during incidents

Manual changes cause the most instances of production drift — an engineer increases an RDS instance size to handle unexpected load, someone modifies security group rules for debugging, or a quick fix during an incident becomes permanent when no one updates the Terraform code.

How to Set Up a Scheduled GitHub Action

GitHub Actions schedule workflows using cron syntax, allowing automated runs at specific intervals. For drift detection, daily checks ensure that unauthorized changes are caught within 24 hours while avoiding alert fatigue from too-frequent notifications.

Implementation Steps

1. **Multi-environment configuration** — Most organizations run separate dev, staging, and production environments, each with its own Terraform state file and AWS account. The GitHub Actions matrix strategy enables you to check all environments in parallel rather than sequentially.
2. **Secure authentication with OIDC** — OpenID Connect eliminates long-term AWS credentials in GitHub. Configure IAM roles that trust GitHub's OIDC provider to establish the connection.
3. **Running Terraform plan with drift detection** — Execute `terraform plan` with flags that enable automated drift detection.

4. **Parsing plan output to get actionable information** — Extract details about drift to make notifications useful. Deletions and replacements indicate more serious drift than updates.
5. **Using GitHub Issues for drift tracking** — GitHub Issues provide persistent tracking of drift over time, creating an audit trail and allowing issues to be assigned to team members for resolution.
6. **Slack notifications** — For immediate visibility, Slack notifications alert the team when drift occurs. Production environments and security-sensitive changes trigger urgent notifications.
7. **Artifact storage** — Terraform plans contain detailed information valuable for debugging and analysis. Preserving this output allows teams to review exactly what changed.
8. **Filtering expected changes** — Some drift patterns are expected and should not trigger alerts. Auto-scaling adjustments and AMI updates commonly create harmless drift.

Key Benefits

Benefit	Description
Zero External Dependencies	No third-party SaaS tools or agents required
Native Exit Code Logic	Leverages Terraform's `--detailed-exitcode` for precise drift detection
Parallel Execution	Matrix strategy enables concurrent checks across multiple environments
Audit Trail	GitHub issues provide timestamped drift history and workflow run links
Secure Authentication	OIDC eliminates static credential management
Cost Effective	Runs on GitHub Actions free tier for small to medium usage

Alternative Approaches

- **Terraform Enterprise** — Built-in drift detection with a paid subscription.
- **Terragrunt** — Extends Terraform with drift detection helpers.
- **dflook/terraform-check** — Pre-built action in the GitHub Actions marketplace.

Why Build Your Own?

Custom drift detection provides control and flexibility that third-party tools might not offer. You can define detection logic that handles organization-specific edge cases, such as ignoring certain drift types, implementing custom severity calculations, or integrating with internal systems. Your infrastructure data stays within GitHub's environment rather than requiring third-party access to state files and cloud credentials.

Considerations for Production Use

- **Multi-Account Support** — Use a matrix strategy with account-specific roles or dynamic role selection based on directory structure.
- **Sensitive Data Handling** — Implement plan output sanitization if your infrastructure includes secrets or sensitive configuration.
- **Issue Lifecycle Management** — Add automation to close issues when drift is resolved.
- **State Lock Handling** — The `-lock-timeout=5m` provides basic protection, but consider monitoring for persistent lock issues.
- **Error Notification** — Add Slack/email notifications for plan failures in addition to GitHub issues.

Conclusion

Terraform drift detection through GitHub Actions solves a fundamental infrastructure management problem without requiring specialized tools or additional costs. The implementation handles the complete detection lifecycle: scheduled workflows check infrastructure nightly, parse Terraform output to identify changes, and route notifications based on severity and environment.

Building your own drift detection requires understanding Terraform's exit codes, parsing its plan output, and orchestrating GitHub Actions workflows. This investment pays off through complete control over detection logic, integration with your existing tools, and infrastructure data that never leaves your GitHub environment.

For teams ready to implement more comprehensive Terraform automation, Terrateam extends beyond drift detection to provide plan and apply automation, cost estimation, and policy enforcement integrated directly with GitHub workflows.

GitHub Actions Workflow for Terraform Project | April 21, 2026

Infrastructure as Code • DevSecOps • CI/CD Automation